

ch3 Intro of SQL

3.2 Data Definition

Create Table

```
1 create table instructor (  
2     ID char(5),  
3     name varchar(20),  
4     dept_name varchar(20),  
5     salary numeric(8,2))
```

Domain Types

- **char(n)**:固定长度

比如用char(10)存储一个字符串“Hello”,那么会自动补充五个空格让它长度为10

- **varchar(n)**:可变长度, 最长为n
- **Int**:整数类型 4字节
- **Smallint**:小的整数, 2字节
- **real**:单精度浮点数, 4字节
- **float**:双精度浮点数, 8字节
- **numeric(p,d)**:d个小数位, p个总位数的一个数字

Integrity Constraints

- primary key
- foreign key... references r
- not null

Modify tables

- `drop table r`:删除表格r
- `alter table r add A D`:添加domain D, 属性为A的列
- `alter table r drop A`:删除A列

3.3 Basic Structure of SQL Queries

```
1 select A...  
2 from r...  
3 where P
```

Select

返回对应的列, 相当于project, 对大小写不敏感

去重: `select distinct name`

选择所有属性: `select * from table`

包含运算符: `select salary/2 from table`

Where

`where dept_name='comp' and salary >10`

可以用 `and, or, not` 也可以比较运算

From

`from A, B` 相当于对AB做笛卡尔积, 也就是把A的每一行都会和B的每一行进行拼接, 结果表的行数=A的行数*B的行数, 为了防止生成无效的组合, 一般用where限制

```
1 select name, course_id
2 from instructor , teaches
3 where instructor.ID = teaches.ID
```

3.4 Additional Basic Operations

Rename Operation: As

可以在 `select, from` 里面用

`select` 是给列起一个新名字, `from` 用是可以在同句的 `select, where` 里面简化名字

```
1 select student.name as student_name, instructor.name as instructor_name
2 from student, advisor, instructor
3 where student.ID = s_ID and instructor.ID = i_ID
```

或者

```
1 select T.ID, T.name
2 from instructor as T, instructor as S
3 where T.salary > S.salary and S.ID = 12121
```

String Operations

函数: `substring, upper, trim`

单引号表示常量, 字符串中用 `'` 转义

对大小写敏感

`percent: %, underscore: _:`

`'%sa%':` 包含sa的字符串

`'Intro%':` 以Intro为开头的字符串

`'__ _':` 三个字符的字符串

`'__ _%':` 长度至少为4的字符串

Ordering the Display of Tuples: Order by

```
1 | select* from instructor order by deptname asc,salary desc
```

先升序排列系名，然后在同一系中降序排列工资

3.5 Set Operations

- `union` 并
- `intersect` 交
- `except` 差

比如

```
1 | (select course_id from section where semester = 'Fall' and year = 2017)
2 | union
3 | (select course_id from section where semester = 'Spring' and year = 2018)
```

3.6 Null Values

`null`代表未知或不存在

任何包含`null`的表达式的结果仍为`null`

任何包括`null`的比较的结果为`unknown`,比如

```
1 | 5<null
2 | null<>null
3 | null=null
```

在 `where` 子句中如果结果是`null`的行(tuple)会被直接过滤

用 `is null` 和 `is not null` 检测空值

3.7 Aggregate Functions

`avg,min,max,sum,count`

- `select avg(salary) from...where...` 返回工资均值
- `select count(*) from...` 返回元组个数
- `select name, min (salary) from instructor` 聚合函数和属性不能同时出现在选择中,除了用 `group by`

Aggregation with Null values

除了`count`,其他都会忽略`null`.

4个非`null`, 1个`null`, `count` 返回5, `avg` 的除数为4

Group By

根据...分组

```
1 | select dept_name, avg (salary) as avg_salary
2 | from instructor
3 | group by dept_name;
```

会展示每个系的平均工资

```
1 select dept_name, ID, avg (salary)
2 from instructor
3 group by dept_name;
```

错误，ID如果要在select里面就必须在group by里面出现

Having

找出平均工资大于80000的系名

```
1 select dept_name
2 from instructor
3 group by dept_name
4 having avg (salary) > 80000;
```

注意这里一定先分组才能使用聚合函数，又因为having是在分组后执行，而where是在分组前执行，所以一定不能用where

3.8 SELECT Statement Processing Order

```
1 SELECT column_list
2 FROM table_list
3 WHERE search_condition
4 GROUP BY group_by_list
5 HAVING having_condition
6 ORDER BY order_list [ASC|DESC];
```

1. where 过滤
2. group by 分组
3. 计算聚合函数
4. having 过滤组
5. order by 排序

Nested Subqueries

```
1 select A1, A2, ..., An
2 from r1, r2, ..., rm
3 where P
```

A_i 可以被一个能产生单值的子查询替换

r_i 可以被任意的一个有效子查询替换

P可以被 $B \text{ <operation> (subquery)}$ 替换，B是属性

Set Membership

`in` 用来检测集合的成员关系，这个集合是子查询的结果

```

1 | select distinct course_id from section
2 | where semester = 'Fall' and year= 2017 and course_id in
3 | (select course_id from section where semester = 'Spring' and year= 2018)

```

等效于

```

1 | (select course_id from section where semester = 'Fall' and year = 2017)
2 | intersect
3 | (select course_id from section where semester = 'Spring' and year = 2018)

```

注意用 intersect 时不用 distinct 是因为在交集时默认去重

not in: 找出2017秋但是不在2018春的课程

```

1 | select distinct course_id from section
2 | where semester = 'Fall' and year= 2017 and
3 | course_id not in (select course_id from section
4 | where semester = 'Spring' and year= 2018)

```

也可以自己设集合（枚举集合）

```

1 | select name
2 | from instructor
3 | where name not in ('Mozart', 'Einstein')

```

Some

```

1 | select name
2 | from instructor
3 | where salary > some (select salary
4 | from instructor
5 | where dept_name = 'Biology');

```

工资比其中至少的一个人高

- "20 = some (10,20,30)" is true
- "20 in (10,20,30)" is true
- "20 <> some (10,20,30)" is true
- "20 not in (10,20,30)" is false

All

比其中的任何一个都...

- 20 <> all (10,20,30) is false
- 20 not in (10,20,30) is false
- 20 = all (10,20,30) is false
- 20 in (10,20,30) is true

all 和 some 都不能用枚举集合

Test for Empty Relations

```

1 select distinct course_id
2 from section as S
3 where semester = 'Fall' and year = 2017 and
4 exists (select *from section as T
5         where semester = 'Spring' and year= 2018
6         and S.course_id = T.course_id);

```

外部查询的名字可用在子查询，这样的子查询叫 correlated subquery 相关子查询

not exist

```

1 select distinct S.ID, S.name
2 from student as S
3 where not exists ( (select course_id
4                     from course
5                     where dept_name = 'Biology')
6                  except
7                  (select T.course_id
8                   from takes as T
9                   where S.ID = T.ID));

```

找到上了生物系的所有课程的所有学生

Test for Absence of Duplicate Tuples: unique

unique 检测子查询是否有重复的元组，如果没有，返回true

```

1 select T.course_id
2 from course as T
3 where unique ( select R.course_id
4                from section as R
5                where T.course_id= R.course_id
6                and R.year = 2017);

```

Find all courses that were offered at most once in 2017

Scalar Subquery 标量子查询

标量子查询只返回一个值

From 里面的子查询

```

1 select dept_name
2 from ( select dept_name, avg (salary) as avg_salary
3        from instructor group by dept_name) as dept_avg
4 where avg_salary > 42000;

```

每个from中的子查询必须重命名一个名字，即使这个不被使用

with 提高可读性

先给子查询起名，临时作为一张表再继续查询

```

1 with dept_sal as
2 (
3     select dept_name, avg(salary) as avg_salary
4     from instructor
5     group by dept_name
6 )
7 select dept_name, avg_salary
8 from dept_sal
9 where avg_salary > 42000;

```

先得到系名，平均工资的一个表格，起名为dept_sal，然后对这个表选出工资满足条件的

3.10 Modification of the Database

Deletion

```

1 delete from r where P

```

r是关系(表),P是条件

```

1 delete from instructor 删除所有
2 delete from instructor
3 where dept_name in (select dept_name
4                     from department
5                     where building = 'watson');

```

Insertion

```

1 insert into r ( A1, A2, ..., An )
2 values ( v1, v2, ..., vn )

```

比如

```

1 insert into course (course_id, title, dept_name, credits)这个括号可以省略
2 values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

```

或者添加多个tuple

```

1 insert into course(course_id,title,dept_name,credits)
2 values('CS-437','Database Systems','Comp. Sci.',4),
3 ('CS-401', 'Data Structure', 'Comp. Sci.', 5),
4 ('CS-412', 'Compiler', 'Comp. Sci.', 3);

```

Updates

```

1 update r
2 set A1 = v1, A2 = v2, ..., An = vn
3 where P

```

把所有人的工资乘1.05

```
1 | update instructor
2 | set salary = salary * 1.05
```

把低于平均工资的人的工资乘1.05

```
1 | update instructor
2 | set salary = salary * 1.05
3 | where salary < (select avg (salary)
4 |                 from instructor);
```

ch4

4.1 Join Operations

结合多个关系

```
1 | select name, course_id from instructor join teaches
2 | on instructor.ID = teaches.ID
3 | 等效于
4 | select name, course_id from instructor, teaches
5 | where instructor.ID = teaches.ID;
```

```
1 | select name, title from instructor join teaches on instructor.ID = teaches.ID
   | join course on teaches.course_id = course.course_id
```

Inner Join

只保留两张表能匹配的行

```
1 | select * from student inner join takes on student.ID = takes.ID
```

inner是可以省略的

Outer Join

不但保留匹配成功的还要保留匹配失败的，比如一个学生没有选课，最后也要留下来，然后选的课会填充为null

- left outer join
- right outer join
- full outer join

这三个的outer都可以省略

Left Outer Join

```
1 | select *
2 | from student left join takes
3 | on student.ID = takes.ID
4 | where dept_name = 'Physics'
```

where表示对结果筛选，而on是表之间连接方式，所以不能用 and 和前面的 on 并列

常用 `left join` 查找没有选课的学生

```
1 select student.*
2 from student left join takes
3 on student.ID = takes.ID
4 where takes.ID is null
```

Right Outer Join

Full Outer Join

4.2 Views

view作为虚拟表，不创建新表，只是保存当前SQL，用来定义view的真实表叫base table基表,view可以当做表来操作

Create a View

```
1 create view view_name (A1, ..., An ) as select_statement
```

如果不标注属性就用原来的属性值

Views Defined Using Other Views

可以用一个view定义另一个view

Benefits of Using Views in SQL Queries

- Customize data
- Focus on specific data
- Combine partitioned data
- Simplify data manipulation
- Enhance data security

Materialized Views

把结果真正保存，适合数据量极大，不频繁更新而且需要频繁查询的情况。因为它不像view一样实时更新，可能会过期

Update of a view

一个view可以更新的条件:

- from 里面只能有一个表
- select 不能有聚合、表达式、`distinct`
- 缺失字段必须允许null
- 不能有group by/having

4.3 Transactions

事务是一组操作，它们共同完成一个完整逻辑功能。

一组必须整体执行的SQL,不能执行一半

结束必须commit完成提交或者rollback撤销事务

Explicit Transaction

明确写 `begin transaction` 和 `commit`

```
1 begin transaction
2
3 update account
4 set balance = balance - 100
5 where name = 'zhang';
6
7 update account
8 set balance = balance + 100
9 where name = 'wang';
10
11 commit
```

Implicit Transaction

一条SQL就是一个隐式事务

4.4 Integrity Constraints

约束通常写在 `create table` 里面

常见约束

- primary key
- no null
- unique
- check(P)
- foreign key

Primary Key

非空且唯一，能唯一确定一个人

```
1 create table department (
2     dept_name varchar(20) primary key,
3     building varchar(15),
4     budget numeric(12,2)
5 );
```

Composite Primary Key (复合主键)

```
1 create table classroom (
2     building varchar(15),
3     room_number varchar(7),
4     capacity numeric(4,0),
5     primary key (building, room_number)
6 );
```

因为单个属性不能保证unique

Not Null

保证变量为非空

```
name varchar(20) not null
```

Unique

`unique ( A1, A2, ..., Am)` 这些属性不能有重复, 比如不能注册同名邮箱两次, 但是可以为空

Check

每个元组必须满足谓词条件

```
1 create table section (  
2   year numeric (4,0) check (year > 1900),  
3   primary key (course_id, sec_id, semester, year),  
4   check (semester in ('Fall', 'winter', 'Spring', 'Summer')))
```

Assigning Names to Constraints

```
constraint minsalary check (salary > 29000)
```

给约束起名, constraint 约束名 check 约束条件

添加或删除条件:

```
1 alter table instructor drop constraint minsalary;  
2 alter table instructor add constraint minsalary check (salary > 20000);
```

Referential Integrity Constraints 外键

外键默认引用对方的主键

可以把多个属性结合成复合外键

```
1 create table teaches (  
2   ID varchar(5) references instructor,  
3   year numeric(4,0),  
4   foreign key (year) references instructor(in_year)  
5   primary key (ID, course_id, sec_id, semester, year),  
6   foreign key (course_id, sec_id, semester, year) references section);
```

Cascading Actions

默认拒绝删除原表数据 (外键引用给子表)

```
1 create table instructor ( ...  
2     dept_name varchar(20),  
3     foreign key (dept_name) references department  
4         on delete cascade  
5         on update cascade, ...)
```

这样写就会自动对子表的数据删除或更新

或者, 写 `on delete set null` 在后面, 当外键被删除时会设为NULL, 更新会设为默认值

```

1 create table instructor (
2     ...
3     dept_name varchar(20) default 'Music',
4     foreign key (dept_name) references department
5     on delete set null
6     on update set default,
7     ...)

```

Integrity Constraint Violation During Transactions 事务中违反约束

如果一个SQL本身是一个事务，那么会全部撤销 roll back

```

1 insert into instructor values ('12121', 'Wu', 'Finance', '90000'),
2 ('15151', 'Mozart', 'Music', '40000'), ('12121', 'Einstein',
3 'Physics', '95000'),
4 ('32343', 'El Said', 'History', '60000');

```

如果在一个显式事务里面，那么只有这个句子会拒绝

```

1 begin transaction
2 insert into instructor values ('12121', 'Wu', 'Finance', '90000');
3 insert into instructor values ('15151', 'Mozart', 'Music', '40000');
4 insert into instructor values ('12121', 'Einstein', 'Physics', '95000'); 拒绝
5 insert into instructor values ('32343', 'El Said', 'History', '60000');
6 commit

```

Try-catch

```

1 begin try
2 begin transaction
3 insert into instructor values ('12121', 'Wu', 'Finance', '90000');
4 insert into instructor values ('15151', 'Mozart', 'Music', '40000');
5 insert into instructor values ('12121', 'Einstein', 'Physics', '95000');
6 insert into instructor values ('32343', 'El Said', 'History', '60000');
7 commit
8 end try
9 begin catch
10 print 'error caught:'
11 print error_message()
12 rollback
13 end catch

```

一旦遇到违约的句子就会跳到catch

4.5 Built in Data Types in SQL

Date and Time Types

- date: date '2005-7-27'
- time: time '09:00:30'
- datetime: datetime '2005-7-27 09:00:30.75'

Functions:

- GETDATE():返回当前系统的日期和时间
- YEAR(date): 返回指定日期的年份(int)
- MONTH(date)
- DAY(date)
- DATEDIFF(date part, start date, end date):返回从开始到结束这段的时间个数, 比如
`datediff(month,hiredate,getdate())` 是得到从雇佣到现在的月数

Type Conversion

Implicit

当两个类型不同, SQL会自动转换, 但是有时会失败

Explicit

`cast(e as t)` 把表达式e转换成类型t

```
1 select name, salary from instructor
2 where cast (salary as varchar(10)) like '8%';
```

找到工资开头为8的人

Default Values

插入元组时如果没有设置该值, 会被设为默认

```
1 create table student
2 (... ,tot_cred numeric (3,0) default 0);
```

Large-Object Types

图片音频视频等等

- blob 二进制, 存储图像视频等
- clob 字符, 存储长文章小说等
book_review clob(10KB), image blob(10MB), movie blob(2GB)
查询大对象时不直接返回对象, 而是返回指针

User-Defined Types

`create type Dollars from numeric (12,2) not null`, Dollar是类型名, numeric 是真正的类型, not null 限制非空, `drop type xxx` 是删掉之前创造的类型

Generating Unique Key Values

定义时加identity, `ID int identity`, 之后插入时不用写ID的值, 会自动递增添加

```
1 insert into instructor (name, dept_name, salary)
2 values ('Smith', 'Comp. Sci.', 100000);
```

命名层级

数据库系统包含许多catalog(约等于数据库), schema相当于目录, 文件夹, object是具体的文件表格

4.6 index creation

索引是一个数据结构, 用来方便检索定位数据, 但是需要写入, 而且占空间
语法:

```
1 create index 索引名
2 on 表名(字段)
```

主键自动创建索引

4.9 Authorization 权限

```
1 grant <privilege list> 权限
2 on <relation or view> 对象
3 to <user list> 用户
```

user list:

- a user-id
- **public**
- A role

比如 `grant select on department to Amit, Satoshi`

Privileges in SQL

- select
- insert
- update
- delete

Revoking Authorization 撤销

```
revoke on from
```

如果包含 `public` 那么除了显式授权的用户其他人都会撤销授权

Roles

防止一个一个用户授权麻烦, 用role授权, 每个人继承角色的权限

```
create role<name> 创建角色
```

```
grant <role> to <users> 授权
```

ch5 Advanced SQL

5.1

5.2 Variables

- Local ~:用户定义的, 用@前置
- Global ~:DBMS定义的, 用户只能读取不能修改, 用@@前置

Local Variables

```
declare @dept_name varchar(20), @num int
```

通常不是全局的

设值:可以用常量, 也可以用select语句

```
1 set @dept_name = 'History'
2 set @num = (select count(*) from student)
```

5.3 Operators

- +, -, *, %
- =, <, >, <=, >=
- NOT, AND, OR
- 字符串拼接+

5.4 Built in Functions

- AVG, SUM, MAX, MIN, COUNT
- cast, convert
- lower, upper, substring
- abs, log, power, cos, sin
- getdate, year, month, day, datediff

5.5 Output Statement

```
print 'hello'
```

5.6 Error Handling

User-defined Error

```
raise error ('输出的字符串', 等级, 状态)
```

5.7 Control Statements

- begin ...end
- if...else if...else
- while
- continue
- break

```

1 while (select avg(price) from book) <= 500
2     begin
3         if (select max(price) from book) > 800
4             break;
5         print 'Raise the prices!';
6         update book set price = price * 1.1;
7     end

```

5.8 Cursor

可以逐行处理数据

1. declare cursor_name scroll(允许上下滚动读取) cursor for 选择语句, 如果没有 scroll 那么只允许 fetch next
2. open cursor_name
3. fetch
4. close cursor_name
5. deallocate cursor_name

处理数据

```
fetch [next|first|last|absolute n] from cursor_name into variable_list
```

下一行/第一行/最后一行/第n行

fetch next from student_cursor into @id, @cred 把下一行对应的数据赋值给对应的两个变量

@@fetch_status 表示操作的状态, 0是成功, -1是失败

修改当前行

```

1 update student set name = 'alice'
2 where current of student_cursor

```

删除当前行

```

1 delete from student
2 where current of student_cursor

```

5.9 Stored Procedures 存储过程

是一组SQL, 存储在数据库里, 更快, 更安全

类型

- System ~: 由DBMS提供的, 以 sp_ 开头的, 可以直接用名字引用
- User ~: 由用户定义, 在数据库存储执行, 没有名字前缀

执行

given a department, find out the average salary of instructors in that department


```

1 create procedure avgsal
2 @dept varchar(20), @av numeric(8,2) out 输入系, 输出av
3 as 具体过程
4     select @av=AVG(salary) from instructor
5     where dept_name=@dept
6 go 结束

```

执行:

```

1 declare @a numeric(8,2)
2 execute avgsal 'Music', @a out
3 select @a

```

删除存储过程

```
drop procedure avgsal
```

Return

```

1 CREATE PROCEDURE check_dept
2     @name varchar(20), @dept varchar(20)
3 AS
4     IF(SELECT dept_name FROM instructor
5        WHERE Name = @name) = @dept
6        RETURN 1
7     ELSE
8        RETURN 0
9 GO
10 DECLARE @status int;
11 EXECUTE @status = check_dept 'Wu', 'Finance';
12 SELECT @status

```

5.10 User-Defined Functions

```

1 create function dept_count (@dept_name varchar(20))
2     returns int
3 begin
4     declare @d_count int;
5     select @d_count =count (*) from instructor
6     where dept_name = @dept_name
7     return @d_count;
8 end

```

调用:

```

1 select dept_name, budget from department
2 where dbo.dept_count (dept_name ) > 2
3 select dept_name, dbo.dept_count (dept_name ) as num from department

```

创建函数的语句必须自己在一个batch

调用函数时至少要用两部分的函数名, 如 `dbo.dept_count`

5.11 Triggers

触发器是在数据被修改时系统自动执行的procedure

可以被 insert, update, or delete 操作调用

必须指定什么时候使用触发器，触发器采取的行动

类型

- After :在SQL执行后触发
- Instead of: trigger替代原SQL

创建触发器

```
1 create trigger trigger_name on table_name
2     {for|after|instead of}{[insert][,][update][,][delete]}
3 as
4     trigger_body
5 go
```

这里for=after

工作原理

每个触发器都有两个逻辑表：inserted and deleted table，各自包含插入的新数据和删除的旧数据，其中**update**更新操作视为插入+删除，也就是**两张表都会新加数据**
触发器可以获取这两张表的数据，但是不能直接修改
比如在真正插入数据之前会将数据更新到inserted表里面然后可以用Instead of检测是否插入表

ch6 Database Design Using the E-R Model

6.1 Design Phases

1. Initial phase: 描述用户需求
2. conceptual-design : 概念模型
3. Final phase: 真正数据库：logical design数据库表的设计和 physical design 数据在磁盘怎么存储

必须避免数据的冗余和不完整

6.2 E-R Model

Entity Sets

entity: 一个存在的对象而且能区别于其他对象

entity set: 同类别的entity,比如学生，导师，航班，课程

一个entity包含一系列的属性attributes,一部分属性构成主键

我们只引入对模型有用的实体和属性

实体集表示

长方形表示实体集，属性在长方形内部，主键属性用下划线标注

Relationship Sets

同类型的关系组成的集合

关系集表示

用菱形表示(diamond),然后用线连接实体集

Relationship Sets with Attributes

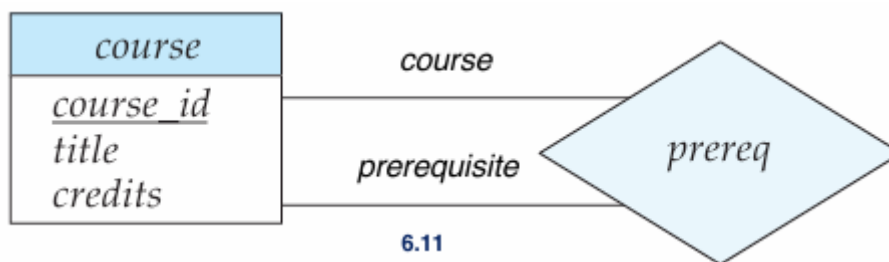
包含属性的关系叫做 descriptive attributes

比如学生和导师之间的关系记录了何时建立关联

Roles

每个实体都在关系集中扮演一个角色，只是一般被隐藏

当同一个实体集以不同角色参与同一个关系集多次时，需要指定这些角色



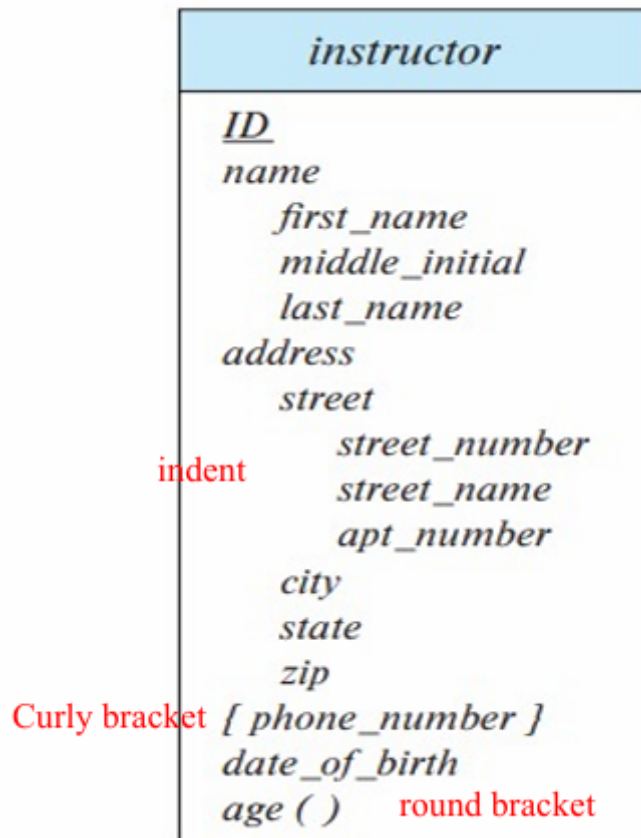
Degree of a Relation Set

关系集连接的实体集的数目

本课只考虑二元关系 binary

6.3 Complex Attributes

- Multivalued attribute: 一个属性包含多个值，比如一个人可以有多个电话号码
- Composite attribute: 可以拆成许多子属性，比如地址可以继续拆成街道城市等
- Derived attribute: 可以通过别的属性计算得到，比如通过生日得到年龄



6.4 Mapping Cardinalities

- One to one
- One to many
- Many to one
- Many to many

多用于二元关系

Representing Participation

- Total participation : 实体集的每个实体都有至少一个关系(每个学生都要找一个导师), 关系和实体集之间用**双线**连接
- Partial participation: 有些实体是空的没有关系, 用**单线**连接

Cardinality Limits

0..*:可以是0到无限

1..1:只能是1

0..1:0或者1

6.5 Primary Key

Relation of Relationship Set

关系集可以用一个关系(表格)表示

关系(表格)中包含相关实体集的主键, 还有关系集的属性叫做descriptive attributes

Primary Key for Relationship Sets

决定属性是描述属性的子集，决定属性构成了关系集主键的一部分

主键的选择取决于mapping cardinality

- Many - Many: 两个实体集的主键和关系集的决定属性discriminator attributes共同构成主键
- One <- Many /Many -> One: 只用many一端的主键
- One<->One: 任何一段都可以被选为主键

Weak Entity Set 弱实体集

有些实体无法唯一存在，必须依赖别的实体

非弱实体就是强实体Strong entity,它们自己有主键可以唯一确定对象

用来识别弱实体的强实体为识别实体集 Identifying Entity Set，它和弱实体的关系为识别关系 identifying relationship

discriminator attributes部分键是弱实体内部用于区分自己的属性，必须在同一识别实体下区分，不能全局唯一，也叫partial key

表示方法：弱实体用双矩形，识别关系用双菱形，部分键用下划虚线

复杂的实体-关系图可能需要拆分为多个页面，一个实体集可能出现在多个页面中。实体集的属性应仅显示一次，即在其首次出现时显示。实体集后续的出现应不显示任何属性，以避免在多处复相同信息，这可能导致信息不一致。

6.7 Reducing E-R Diagram to Database Schema

强实体的表示

用相同的属性和主键

弱实体的表示

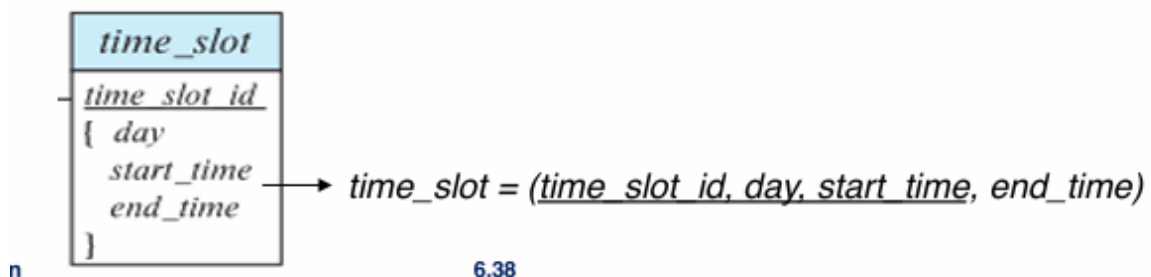
要用识别实体的主键+弱实体的部分键组成弱实体的属性

组合属性的表示

把composite属性拆开，derived属性不直接表示，会在view中表示

多值属性的表示 Multivalued

把多值属性拆开，和原来的主键组合成新的关系，然后这个整体作为新的主键



关系集的表示

多对多

单独建表R

属性包含两个实体集的主键和关系集的所有属性descriptive attribute

R的主键是两个实体集的主键+关系集的决定属性 discriminator attribute,同时这两个实体集的主键是R的两个外键

一对多/多对一

把一的主键和关系集的所有描述属性加到加到多的那一边构成R, 那么一的主键是R的外键

一对一

任意一边加入另一边的主键作为外键和所有描述属性

ch7 Normalization

7.1 Data Redundancy

7.2 Decomposition

避免信息重复的唯一方式就是把它分解成更小的关系模式

If the schemas R1 and R2 form a decomposition of R, then $R = R1 \cup R2$

不是所有的分解都是好的, 有的时候会分解得到重复的属性, 这就导致了lossy decomposition

如果分解后的两个关系的笛卡尔积和原来的关系相等, 那么称为**lossless decomposition**

相反, 如果原来关系变成笛卡尔积的子集, 称为**lossy decomposition**

Normalization

根据函数依赖把坏表做无损分解成好表(不冗余)

Functional Dependencies

是现实世界中的一系列规则, 如果一个关系实例满足这样所有的规则, 就叫做legal instance

规则比如: 学生和导师都有唯一的ID, 只有一个名字, 只属于一个系, 每个系都有自己的一个预算

7.3 Functional Dependency

当且仅当每个X的值都精确对应一个Y值, 就叫这个属性Y is **functionally dependent** on a set of attributes X, 记作 $X \rightarrow Y()$

其实就是函数, 多个x的函数y值可能相同, 但是一个x不能有多多个y

当这个x能确定的属性从y扩大到整个元组时, x就变成了主键

所有属性都 functionally dependent on candidate keys,当然包括super keys

Super key

└── Candidate Key

└── Primary Key

严格定义

a,b是关系模式R的两类属性, $a \rightarrow b$ 当且仅当 $t_1[a] = t_2[a] \Rightarrow t_1[b] = t_2[b]$

函数依赖和键

K is super key 当且仅当 $K \rightarrow R$

K is candidate key 当且仅当 $K \rightarrow R, \text{no } \alpha \subset K, \alpha \rightarrow R$

函数依赖比键更一般更强大，键强调唯一的元组，函数依赖强调属性之间的决定关系

用途

检测关系 r 在函数依赖集 F 下是否合法，合法就说 r **satisfies** F ，当前的具体数据满足

F **holds** R 是关系模式 R 满足函数依赖集 F ，是这个表永远遵守的规则

Trivial Functional Dependency

$\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

一般不会提供什么有用的信息

Full Functional Dependency

Y 全依赖于 X 当且仅当 Y 依赖于 X 并且不依赖于 X 的真子集，也就是 X 是最小的相反的，叫做 **partial** ~

Transitive functional dependency

$X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$

(Non-)Prime Attributes

在候选键中的属性是 prime attributes, 不在的是 non-prime

7.4 Schema normalization

最大程度减少数据冗余

Normal Forms 范式

用于判断关系模式是否处于好的模式，规范化是将数据库模式设置为更高的范式的过程
如果关系不符合良好模式，那么把它无损分解为小的关系模式，每个都是一个合适的范式

1NF

每个属性只有一个值(单值)，没有重复的元组

2NF

$R \in 1NF$ 前提下，所有 non-prime 属性 **全依赖** 于每个 R 的候选键

消除非主属性的部分依赖

- Borrow (StuID, Name, Dept, Mngr, BookID, Date) $\notin$ 2NF
 - StuID $\rightarrow$ Name
 - StuID $\rightarrow$ Dept
 - Dept $\rightarrow$ Mngr
 - StuID $\rightarrow$ Mngr
 - BookID $\rightarrow$ Title
 - (StuID, BookID) $\rightarrow$ Date
- Name, Dept, Title and Mngr are partially functionally dependent on the candidate key

上面例子的候选键是(StuID,BookID), 但是name不全依赖于候选键, 可以直接由更小的部分确定, 所以不属于2NF

Normalize

把表拆开, 谁决定谁就单独成表, 然后那个全依赖于候选键的非主属性留下, 其他的和候选键一部分属性单独成表

3NF

$R \in 1NF$ 前提下, 每个非主属性都不传递性依赖于候选键
 满足3NF也一定满足2NF, 消除非主属性的传递依赖

Normalize

设 $R(\underline{X}, Y, Z)$ 满足传递 $X \rightarrow Y, Y \rightarrow Z$
 就需要拆成两个表: $R_1(\underline{X}, Y), R_2(\underline{Y}, Z)$

BCNF

必须满足每个 $X \rightarrow Y$, X是候选键或超键, 在3NF基础上消除主属性的部分依赖或传递依赖

Functional-Dependency Theory

Closure of a Set of Functional Dependencies

给定一组依赖关系, 有些关系是可以根据这组推导得到的
 就比如 $A \rightarrow B, B \rightarrow C$, then $A \rightarrow C$, 把这些关系全都放在一起叫做**closure** of F, 记作 F^+
 我们可以不断应用Armstrong's Axioms得到F的闭包

- reflexive rule: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
- augmentation rule: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
- transitivity rule: 传递

sound and complete

推论:

- Union: if $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$, then $\alpha \rightarrow \beta\gamma$
- Decomposition: if $\alpha \rightarrow \beta\gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$
- Pseudo transitivity: if $\alpha \rightarrow \beta$ and $\beta\gamma \rightarrow \delta$, then $\alpha\gamma \rightarrow \delta$

Closure of Attribute Sets

由属性集合A能推出的所有属性构成的集合叫做属性集合的闭包 A^+

还可以进一步判断该属性集合是否为超键和待选键

Canonical Cover 最小覆盖

最简的函数依赖集合，不改变原来含义，也就是 $F_c^+ = F^+$

Extraneous Attributes

$AB \rightarrow CD$, 删掉左边的约束会变强，删掉右边的会变弱

检测方法：

- 删掉左边：根据现有的F检测能否推出 $A \rightarrow CD$
 - 删掉右边：F 发生改变，根据F检测能否推出 $AB \rightarrow C$ (删掉C)
- 计算canonical cover：重复以下步骤： $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1\beta_2$, 检测两边是否有多余元素

检测无损分解

分解 $R = (R_1, R_2)$ ，把两个结果做交集，然后用这个集合推出闭包，看R1或R2是否包含在这个闭包，如果至少包含了一个，那么这个分解就是无损的

Dependency Preservation

分解之后不需要重新join就可以检测DF

比如 $R(A, B, C), A \rightarrow B, B \rightarrow C$, 然后分解成 $R_1(A, B), R_2(B, C)$, 这样就是依赖保护

$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$

restriction: F^+ 中只包含 R_i 的属性

算法

$\alpha \rightarrow \beta$ 是否 preserved

先令 $result = \alpha$, 然后对每个子表 R_i 取 R_i 和 $result$ 的交集，然后对这个交集在子表内求闭包，更

新 $result$, 直到 $result$ 不变，然后看如果 $\beta \subseteq result$, 那么就是 preserved

Algorithm for Decomposition Using Functional Dependencies

BCNF 分解算法

找到一个违反BCNF的依赖： $\alpha \rightarrow \beta$ ，即a不是超键，然后把 R_i 拆分成 $R_i - \beta$ 和 $\alpha\beta$

3NF算法

先求 F_c 最小覆盖

对 F_c 每个关系建表

检查候选键，如果没有表包含候选键，那么加一个是某个候选键的表格

删除冗余表，比如存在 $R(A, B), R(A, B, C)$ 那么删掉 $R(A, B)$

ch17 Transactions

17.1 Transaction Concepts

一个事务是一系列的数据库操作

数据库操作主要有两种：read(X):把数据库的数据传给变量X,write(X):把变量X的数据传给数据库，除此之外，每个事务必须以commit或abort(SQL中用rollback)结尾

事务管理不考虑非数据库操作

ACID

- Atomicity: 要么全做，要么全不做
- Consistency: 事务执行前后数据库都要满足正确的规则，比如转账的双方的总金额不能变化，主要由程序员负责
- Isolation: 多个事务同时执行，不能互相看到对方中间结果，只能看到执行前或后
- Durability: 一旦commit成功，结果必须永久保存(即使系统在之后崩溃)

Transaction State

- Active: 事务正在运行
- Partially committed : 最后一个普通操作执行完，正准备提交但是没有真正提交成功，在最后一个操作和commit之间
- Committed: commit操作已完成
- Failed: 在运行一半(active)或者最后提交前(partially committed)
- Aborted: 所有事务都被撤回
- 如果一个事务处于aborted或者committed，那么就说**terminated**

17.3 Transaction Management

为了保证事务执行完数据库仍是正确、一致的

- **Concurrency control scheme:** 实现事务的Isolation
- **Recovery scheme:** 实现Atomicity,Durability

一致性是最终目标。有了事务管理，程序员不用处理并发和崩溃问题

17.4 Schedules

用来指定并发事务的执行顺序

必须包含所有事务的所有操作

必须保持每个事务内部的原有顺序

Serial Schedules

事务执行one by one

保证了一致性，但是效率低

Non-serial Schedules

可能无法保证一致性，导致dirty write/read，但是也有可能成功，对应某个serial schedule

17.5 Serializability

如果一个non-serial schedule效果等价于某个serial schedule 那么就是serializable

Conflict Operation

两个操作冲突，要满足：属于不同事务；访问同一项Q；至少有一个write(Q)

操作 I	操作 J	是否冲突
read(Q)	read(Q)	不冲突
read(Q)	write(Q)	冲突
write(Q)	read(Q)	冲突
write(Q)	write(Q)	冲突

不冲突的操作可以交换

Conflict eq: 如果一个 schedule 可以通过不断交换“不冲突的相邻操作”变成另一个 schedule，那么这两个 schedule 是冲突等价的

Conflict serializable: 如果一个 schedule 可以通过交换不冲突操作，变成某个串行调度，那么它就是冲突可串行化的

但不是所有 serializable 的 schedule 都是 conflict serializable

17.6 Recoverable schedules

cascading rollback : 一个事务失败导致其他一连串事务也回滚

recoverable schedule: 谁读了别人的数据，就要等别人先commit,自己才commit

cascadeless schedule: 比recoverable schedule更严格，那就是 T_1 要读 T_2 写过的数据，那么 T_2 必须已经commit

ch18 Concurrency Control

18.1 Concurrency Control Concepts

保证所有调度都是serializable,recoverable,cascadeless

并发控制不会逐个检查每个schedule，而是使用 concurrency control protocol,在执行过程中就制定规则，从一开始就阻止不安全的schedule出现

18.2 Lock-Based Protocols

锁是用来控制多个事务同时访问同一个数据项的机制

- Exclusive lock : 可以读也可以写, lock-X(A)
- Shared lock: 只能读, lock-S(A)
- 以上两个表示事务请求对数据项A加锁，由并发控制方案检查并批准
- unlock(A) 表示访问完后释放锁

Lock-Compatibility 兼容性

请求的锁	已经有 S 锁	已经有 X 锁
S	true	false
X	false	false

false的一类只有等到别人 unlock 之后才能获得锁

这是数据库内部发生的过程，是系统自动加的，不用自己写

Locking protocol

规定加锁和解锁的顺序

legal schedule: schedule的所有事务都遵守了locking protocol,它保证了serializability

Two-phase Locking Protocol

把每个事务加锁解锁的过程分成两个阶段: growing,shrinking

- growing phase: 只能不断申请锁，不能释放锁
- shrinking phase: 一旦开始解锁就进入这个阶段，只能释放，不能再申请
- lock point: 一个事务获得最后一个锁的那个时刻，如果T1的lock point早于T2,那么可以等价成 $T_1 \rightarrow T_2$,保证 conflict serializable

但是普通2PL不一定避免脏读脏写，因为它允许事务在commit之前unlock

Strict 2PL: 所有的X锁保留到提交或回滚之后释放

Rigorous 2PL: 把所有的X锁和S锁在提交或回滚之后释放

Lock Conversion

Upgrade: $S \rightarrow X$,只能在growing phase里面用

Downgrade: $X \rightarrow S$,只能在 shrinking phase 里面用

这种可以保证serializability前提下提高并发性，比如可以先申请S锁让多个事务一起读取，然后再把后续需要写的事务进行转换

Starvation

读事务一直插队，导致写事务等不到X锁

解决方法：在检查当前是否有其他事务持有锁的同时，还要保证不要被新的事务插队，比如正在X锁等待一个S锁，然后新来了一S锁，正常按照S可以并发应该直接批准新来的S，但是这样造成了插队现象，所以不能批准新来的S

18.3 Implementation of Locking

Automatic Acquisition of Locks

- read(Q): 自动在前面加申请S锁
- write(Q): 先检查是否有Q的S锁，如果有就在前面 upgrade ,没有就申请X锁
- commit/abort之后: 自动释放所有锁

Lock Manager

消息	含义
lock-request	事务请求某个锁
lock-grant	锁管理器批准锁请求
unlock-request	事务请求释放锁
ack	对释放锁的确认
roll back	如果发生死锁，要求某事务回滚

Lock Table

锁表是一个哈希表，每个位置是一个链表，链表记录每个的请求

(Un)Lock Requests Processing

当新请求到达时，manager会把它加到链表末尾，然后判断是否批准

如果当前没有被锁，直接批准；如果已经被锁，需要检查是否和当前锁兼容且前面的请求是否已经被批准（防止饥饿）

所有锁在事务完成后释放，释放后manager会删除记录然后检查后面是否能批准

18.4 Deadlock handling

存在一组事务，组中每个事务都在等另一个事务释放锁，2PL无法阻止死锁出现

必须牺牲一个事务，回滚释放

Deadlock Prevention

从规则上保证系统永远不会进入死锁

- pre-declaration: 在事务执行前把所有需要的数据锁住，但是并发性很低
- Preemptive抢占式：如果老事务需要的锁被年轻的持有，就让年轻的回滚
- Non-preemptive：年轻事务不等老事务，年轻事务需要的被老的持有，年轻自己回滚
- Timeout: 等一段时间还没拿到锁就回滚这个事务，但是可能误判

Deadlock Detection and Recovery

wait-for graph

顶点是事务，边是 $T_1 \rightarrow T_2$,表示T1在等待T2释放锁

死锁 $\Leftrightarrow$ 图中存在环

周期性检测图是否存在环

Deadlock Recovery

选择成本最低的事务作为victim回滚释放锁

- Total Rollback: 把该事务的所有锁都释放，过程简单但是损失较大
- Partial ~: 回滚到刚好足够释放其他事务正在等待的锁，损失小但是实现复杂

ch19 Recovery System

19.1 Failure Classification

System crash

系统因为硬件故障、数据库软件 bug、操作系统 bug 等原因突然崩溃,导致数据不一致
用recovery scheme **恢复机制**处理

Disk failure

导致数据库损坏, 通过**stable storage** 解决(**RAID**:把数据冗余地存到多个磁盘)

Disaster

更大规模的灾难

靠**备份恢复 backup**

19.2 Recovery scheme

恢复模式是数据库系统内部部分, 它通过将数据库恢复到故障发生前的一致状态, 从而确保事务的原子性和持久性。

分成两个部分: 在数据库正常运行时, 提前记录一些信息, 保证崩溃时恢复; 在崩溃后恢复信息

Data Access

数据库不是按条读取数据, 而是按块**block**

- Physical Block: 磁盘上的块, 是真正永久存储的数据
- Buffer: 是内存的缓存区, 保存磁盘的副本, 访问这里更快
- Buffer Block: 是缓存块, 暂时存储数据
- Input/ Output: In是把磁盘数据输入到缓存, Out是缓存写入磁盘

每个事务有他的私人工作区, **read(X)**先检测X是否在缓存区内, 如果没有先调用input(Bx)把X放到缓存, 然后再把缓存处的X赋值给事务变量x; **write(Y)**也是检测Y是否在缓存, 不在就复制, 然后把变量y复制给缓存区

也就是说, write/read是事务和缓存之间的操作, input/output是缓存和磁盘之间的操作

19.3 Log-Based Recovery

恢复的最常用的方法

数据库日志是一串日志记录, 记录了数据库的更新信息

为了日志记录能用于修复, 必须保存在stable storage

类型: (用尖括号包围)

- <T start>: 事务已经开始
- <T commit>: 事务已经提交
- <T abort>: 驳回
- <T,X,V1,V2>: 事务T对数据对象X进行写, 之前的值为v1,写后的值为v2

在日志中储存的数据可能非常大，这些可以在适当的时候删掉

Recovering from failure

如果日志包含start和commit/abort，那么这个事务要重做

如果只有start没有commit/abort，那么恢复时不用做

redo用在已提交的事务，恢复时从头开始

undo用在未提交的事务，恢复时从尾巴开始

在**undo**过程中，每次X恢复到原来的值V就会生成一个 $\langle T_i, X, V \rangle$ 的特殊日志，叫做undo-only日志记录，当undo结束后会生成一个 $\langle T_i \text{ abort} \rangle$ 的日志记录

Checkpoints

重做整个事务会很费时，我们可以不用重做那些已经输出给数据库(磁盘)的事务

所以定期执行checkpointing，所有的更新停止，把所有缓存块输出到磁盘，写一个 $\langle \text{checkpoint } L \rangle$ 的记录日志，L是当前所有事务的列表

当发生crash时候，系统检查日志的最后一个checkpoint，只有在这个L中的事务和之后的事务需要undo/redo

19.4 Backup and Restore 备份和恢复

backup是在数据库做一份数据的复制用来恢复那个disaster的数据，两份数据应该储存到不同位置

restore是用备份把数据库恢复到创造备份时的那个状态

Types of backup

- Complete Backup: 复制数据库的所有数据，占据磁盘空间大，速度慢
- Differential Backup: 复制那些在上次完全备份之后的创造或改变的数据，空间小速度快
- Log Backup: 在上个备份之后备份日志，执行更频繁

Policy

多种备份一起运用

典型的一个policy:

- 每周完全备份
- 每天差分备份
- 每隔两小时日志备份

Restore

- 先保存从故障前最后一次日志备份到故障发生一段的日志，记作**tail-log backup**
- 恢复离故障发生最近一次(最后一次)的完全备份--**complete restore**
- 恢复最近一次的差异备份--**differential restore**
- redo所有在最后一次差异备份之后的日志备份中的事务--**log restores**

ch13 Data Storage Structures

File Organization

数据库是文件的集合，每个文件是一系列records，一个记录是一系列字段fields

为了简化，假设记录的长度固定，一个文件只存一种记录，不同关系使用不同文件，那么可以把文件看做一个**表格**，一个**记录就是一行(元组)**的内容，里面的各种**属性就是字段**。

又假设记录必须小于block

disk block用来读写数据，一般4-16 KB,太小会频繁读写，太大浪费空间

Fixed length Records

如果n是一条记录的长度(size)，那么第i条记录从字节 $n*i$ 开始

不能允许记录跨block

当删除记录i时有三个选择：

- 把records(i+1,n)移到(i,n-1)
- 把record n移到 i，直接顶替掉
- 不移动记录，记录已删除的记录位置，形成一个链表free list，当插入新纪录时优先插入这些位置

Variable Length Records

三种情况导致 Variable Length：

- 一个文件中含有多多种类型的记录
- 字段本身长度可变(varchar)
- 记录的类型匀速重复字段

属性是按顺序储存的，所以把可变长度的属性那一块用(offset开始的字节,length长度)这一组数代替，把真正的数据放在整个固定长度属性的后面，然后用null value bit-map表示哪个属性为空，空属性记作1，非空为0，这样就不用在后面额外写空属性为NULL了。这个bit-map放在固定属性和可变属性之间

Organization of Records in Files

数据库把表中真实的数据存到磁盘中用什么结构

Heap

把记录放在任意的空闲空间，插入快，查询慢

Sequential File

按照search key 存储

删除某条记录时，用pointer chains 类似于链表

插入记录时，如果块内有空位直接插入，没有空位就在overflow block中插入这条数据，同时更新指针链

Multi-table Clustering File Organization

把经常一起访问的多个表数据放在同一个磁盘块里，减少磁盘访问次数

适合混合查询但是不适合单个查询，会导致记录长度不固定，可以用pointer chain连接特定关系的记录

B+ tree

插入删除后仍平衡

Hashing

对搜索键做哈希运算

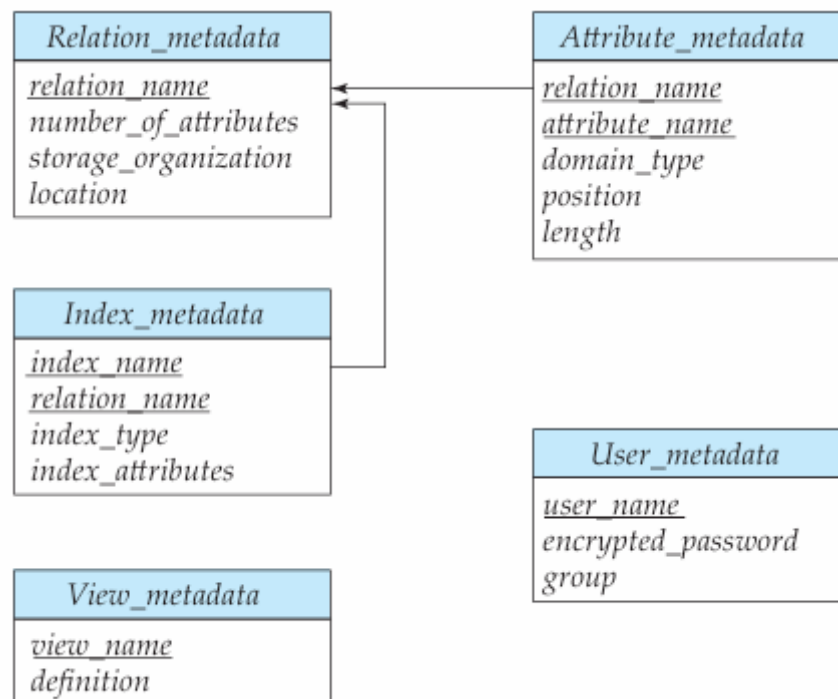
Partitioning

可以把表格分成更小的关系独立存储

查询时默认把所有的关系全扫描，除非用 `where` 限定，比正常的查询速度要快

Data Dictionary

用来存储metadata，描述数据的数据，包括表名，属性的信息，view信息，约束；用户信息包括用户名，密码，权限；统计信息包括记录的条数等；物理存储信息；索引信息



他们之间的关系如上

Column Oriented Storage

每个列独立存储为一个关系(表格)

减少了磁盘的I/O

但是tuple reconstruction会很麻烦，更新删除tuple也会麻烦

事务处理(对于行的处理)不适合

Storage Access

数据库如何访问磁盘？

块 (Block) 是数据库存储和传输的基本单位,要尽量减少内存和磁盘的传输次数

这就是Buffer的作用，Buffer Manager是负责管理buffer的

Buffer Manager

读取数据时，先访问buffer manager，如果块已经在buffer中，直接返回内存地址；如果不在，要先分配缓存空间：踢出一些块，而且只有当这个块被修改时才写回磁盘再替换，之后读取新块并返回地址

在块正在被使用时，不能被写回磁盘。因此在读写数据前pin,读写数据块后unpin,处于读写的数据块为pinned block,有可能有多个操作同时使用，这需要pin count记录，当且仅当pin count==0时才能be evicted

Shared and exclusive locks on buffer,与之前锁的规则一样

Buffer Replacement Policies

least recently used LRU strategy

把最近最久没有访问的块替换掉

Toss-immediate

处理完该块的最后一个信息就立刻丢掉，适合顺序扫描

Most recently used (MRU) strategy

buffer满了之后，删掉最近一次访问的块

```
*create trigger retake on takes
*  instead of insert
*as
*  if exists (select * from takes, inserted
*             where takes.ID = inserted.ID and
*                   takes.course_id = inserted.course_id and
*                   (takes.grade is null or takes.grade <> 'F'))
*    print 'students can only retake a course if they fail in
this course'
*  else
*    insert into takes select * from inserted
*go
```

